mandate

Technical Documentation

Architecture, protocol specification and evaluation

2026 FinTechathon Topic F: Agent Wallet

Mandate Labs

Nanyang Technological University, Singapore

Member	Role	Programme	Study year
Mengzhong Ma	Leader	Computational Finance	PhD, year 4
Sat Chidananda	Member	Computer Science	PhD, year 1
Vrushank Ajay Ahire	Member	Computer Science	PhD, year 1
Wilfried Ariel Mulyawan	Member	Computer Science	PhD, year 1

Mengzhong Ma	mengzhon001@e.ntu.edu.sg
Sat Chidananda	SAT001@e.ntu.edu.sg
Vrushank Ajay Ahire	vrushank001@e.ntu.edu.sg
Wilfried Ariel Mulyawan	wmulyawa001@e.ntu.edu.sg

Advisor: Bao Te, Nanyang Technological University.

Correspondence: Mengzhong Ma, team leader.

Source code: https://github.com/MandateLabs666/mandate-wallet

Contents

Abstract **3**

1 Product scope and research contribution **3**

1.1 The user problem and a concrete task . . . 3

1.2 What the release contributes . . . 3

1.3 Related work and positioning . . . 4

2 Architecture and trust boundaries **4**

2.1 Owner plane, agent plane and chain plane . . . 4

2.2 Deployment and local product components . . . 5

2.3 Data handling and isolation . . . 6

3 System model and security objectives **7**

3.1 Entities, state and adversary . . . 7

3.2 Assumptions . . . 7

3.3 Objectives and non-objectives . . . 7

4 Authorization objects and payment protocol **8**

4.1 PolicySpec and the actual owner authorization path . . . 8

4.2 Canonical requests, terms and deterministic nonces . . . 8

4.3 Typed messages and domains . . . 9

4.4 Registration, signature release and settlement . . . 9

4.5 Conservative budget accounting and safe release . . . 11

4.6 HTTP request binding and compatibility boundaries . . . 12

5 Durable execution and recovery **12**

5.1 Persisted identity and concurrency control . . . 12

5.2 Outcome state and canonical reconciliation . . . 12

5.3 Paid failure, expired commitment and administrative recovery . . . 13

6 Evidence protocol and independent verification **13**

6.1 Data objects and stage association . . . 13

6.2 Canonical salted field commitments . . . 14

6.3 Verifier algorithm and evidence grades . . . 15

6.4 Selective disclosure and privacy interpretation . . . 16

7 Security properties and host-interference analysis **17**

7.1 Budget invariants and concurrent requests . . . 17

7.2 Replay, version changes and signature binding . . . 17

7.3 Revocation, expiry and release ordering . . . 18
7.4 Session evidence integrity and the missing-tail limit . . . 18
7.5 Reference controller, grinding and suppression . . . 18
7.6 Research scope and remaining work . . . 19

8 Evaluation design and measured results 19
8.1 Questions, configurations and trial controls . . . 19
8.2 Gas and latency measurement . . . 20
8.3 Explicit invalid inputs and stateful attacks . . . 20
8.4 Clean reproduction and review scope . . . 21

9 Public Base Sepolia execution case 21
9.1 Roles, account and scope . . . 21
9.2 Transactions and separate outcomes . . . 21
9.3 Independent evidence status and finality interpretation . . . 22

10 Release boundaries and next implementation work 23

A Contract and MCP interface reference 23
A.1 Account methods and observable errors . . . 23
A.2 MCP tools and effect boundaries . . . 24

B Reproduction and deployment reference 24

Abstract

Mandate Wallet combines owner-approved financial delegation, x402 service procurement and separately verifiable purchase evidence. A fixed-owner, fixed-asset smart account atomically reserves shared task and session budgets before a client exposes a token-payment signature. The selected token calls the account's ERC-1271 validator at settlement, so revocation and policy replacement affect outstanding unpaid authorizations. A local gateway presents bounded MCP tools to Claude Desktop, Claude Code and Codex, while the owner reviews administrative transactions in an external wallet.

This document specifies the deployed route, its message domains and state transitions, the durable recovery implementation, and the exact distinction between a signed observation and independently established source truth. It develops the three research directions from the team's previous proposal into an implementation-oriented account of policy-bound actions, delegated-session accountability and host interference. Formal properties are accompanied by assumptions and manual arguments rather than a claim of machine verification.

The public demonstration uses native Claude Desktop, Rabby and Base Sepolia. It pays 0.01 test USDC once, recovers an injected service failure using the same order and settlement, and rejects an unused step after owner-signed session revocation. A separate evaluation records 120 eventual HTTP tasks, 132 settlements without duplicate payment, and 10,000 distinct explicitly invalid contract calls. The independent public-case verifier passes its required financial checks but retains an incomplete overall report. Independent HTTPS provenance, complete unobserved history and isolation from the local administrator remain outside the demonstrated guarantee.[8, 9, 12]

1 Product scope and research contribution

1.1 The user problem and a concrete task

A tool-capable agent needs a way to purchase a service without receiving unrestricted control over its owner's assets. A prompt can describe a budget, but the financial boundary must survive a host that ignores that prompt or calls the signer directly. Mandate therefore places the amount, recipient, approved-step and lifetime constraints in the purchasing account. Tool permissions and connection credentials govern access to the local application, while chain state governs spending authority.

The running task asks Claude to purchase an ETH statistics sample and produce a short brief. The root budget is 150000 atomic USDC units, the per-payment maximum is 100000, and the sample costs 10000. At six decimals these correspond to 0.15, 0.10 and 0.01 test USDC. ETH names the dataset, not the asset being transferred. The fixed result is `datasetId=mandate-fixed-fixture-v2`, `asset=ETH`, `count=3`, `mean=110`. It has no market timestamp, price unit or live-price guarantee.[8]

The owner creates the root policy and a finite session in the workspace through Rabby. Claude reads the account context, obtains the approved service's quote and calls the bounded purchase tool. After payment, the merchant deliberately returns HTTP 500. A second call recovers the original paid order and returns the sample. Finally, the owner revokes the session and the next approved-but-unused service step fails before registration. This one case exercises authority, settlement, delivery, recovery and withdrawal of future permission.

1.2 What the release contributes

The implementation brings together a two-level conservative reservation ledger, registered-digest-only contract signatures, business-step idempotency across agents and policy versions, durable

x402 recovery, native agent integration and a verifier with explicit evidence grades. The engineering contribution lies in this concrete combination and its reproducible failure cases. The document does not claim that scoped smart accounts or delegated wallets were invented here.

The previous proposal organized research around policy-bound execution, minimum delegated-session evidence and host-interference bounds.[7] The release instantiates these directions with a narrower procurement route. A research objective in that proposal is not evidence that a corresponding mechanism has been deployed. In particular, a mandatory on-chain session-root proof, a universal legacy-web proxy, independent paid-merchant TLS attestation, globally enforced model-query limits and beacon-bound anti-grinding are not current release features.

1.3 Related work and positioning

Mandate's selected service-payment route connects typed registration messages to the token's contract-signature callback. The precise token implementation matters because a transfer-authorization interface alone does not ensure contract-signature support.[4, 5, 20]

Scoped agent accounts already exist. Safe documents agent spending limits and multiple-agent configurations, while MetaMask documents granular delegated permissions.[13, 16] These capabilities preclude a blanket assertion that competing wallets enforce every policy only off chain. The comparison in this project is therefore a controlled EOA/Mandate experiment, not a market-wide security ranking.

Acharya studies agent authenticity and transaction intent using a proposed hybrid framework.[1] BAID concerns user–code binding and reports an implemented prototype using code-level authentication.[6] Identity and execution provenance are complementary to Mandate's finite payment scope. The release does not implement their proof systems or derive its security from agent identity registries. ERC-8004 identity metadata is a possible future integration, not an input to the present authorization predicate.[15]

TLSNotary supplies a possible independent source-evidence primitive, while drand supplies publicly verifiable external randomness.[3, 17] Neither substitutes for request binding, budget accounting or evidence retention. The release's optional TLSNotary experiment concerns a separate public API with a same-host notary/prover. Beacon-bound commitments remain research work.

2 Architecture and trust boundaries

2.1 Owner plane, agent plane and chain plane

The owner plane comprises the workspace, onboarding service and external signing wallet. The workspace constructs an unsigned intent, displays the target chain, account, method and scope, and observes its resulting transaction. An intent is not applied authority until a matching successful receipt and contract event establish the change. The trusted owner must review the decoded transaction rather than infer its effect from a page title.

The agent plane contains the MCP transport, fixed connection profile, purchase client, session signer and durable store. The profile binds a particular chain, wallet, root, session and configured services. A connection token authenticates access to that profile. The production gateway does not load the owner private key. A registration relayer submits session-signed registrations and pays native gas. The facilitator submits the token transfer. Holding either gas key does not create owner authority.[11]

The chain plane contains an immutable `MandateAccount`, a deterministic deployment factory and the selected token. The account fixes owner, asset, deployment chain and token EIP-712 domain at construction. The factory exposes deterministic prediction and idempotent account creation. The

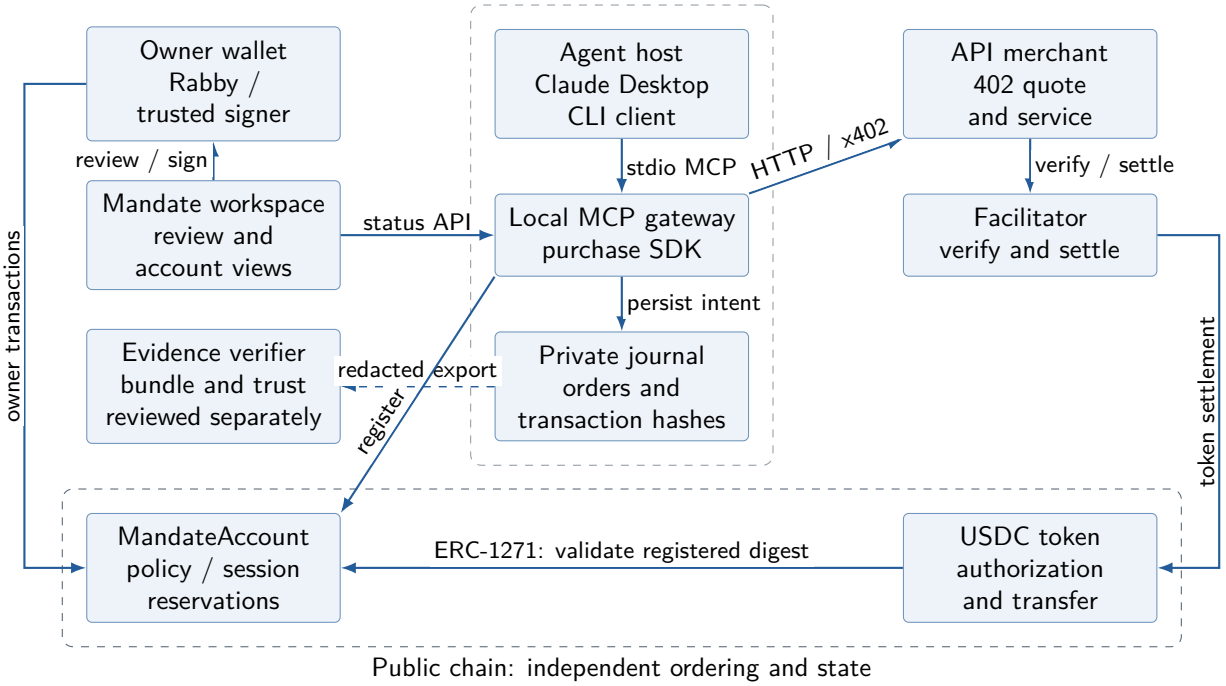


Figure 1: System architecture and the main data and authority paths. The dashed local-host region does not provide isolation against its administrator. Owner transactions and session-signed purchase operations carry different authority. The separate verifier needs an independently reviewed trust file.

account has no agent-facing arbitrary call executor or upgrade administrator. Owner withdrawal remains possible and does not escrow the cash represented by reservations.

The merchant and facilitator are external roles in the architecture, but controlled team services in the demonstrated run. The merchant checks the HTTP request against the registered commitment and returns service data. Its facilitator prevalidation and settlement are different steps. An independently deployed third-party combination requires its own compatibility and request-binding acceptance.

2.2 Deployment and local product components

Component	Implementation responsibility	Principal source
Account / factory	Root and session scope, reservations, signature validation, release	`mandate/contracts/src/`
Purchase layer	Quote checks, original-order recovery, merchant and facilitator fixtures	`mandate/services/`
Agent integration	MCP tools, fixed profile, safe public outputs	`mandate/gateway/`, `mandate/desktop/`
Owner application	Onboarding, external-wallet intents, task and outcome views	`mandate/onboarding/`, `mandate/web/`
Evidence layer	Checkpoints, ActionProof, selective disclosure, verification and controller	`mandate/evidence/`
Evaluation / operations	Invalid inputs, stateful attacks, monitoring and fault recovery	`mandate/research/`, `mandate/operations/`

Table 1: Component-to-source mapping for the accepted code snapshot.

The source-mode onboarding component supports separate Local and Testnet profiles. Local practice starts an isolated Anvil chain and mock token without requiring a personal wallet. Testnet mode prepares deployment, token funding, relayer gas and finite permissions for an external owner wallet. Independent profile directories prevent a newly prepared account from silently replacing an installed agent binding. Restarting guide progress preserves accounts, orders and on-chain authority.

The Guided Claude Desktop package points to the selected local connection and has no required manual profile fields. The user imports and enables it in Claude's graphical extension settings. The advanced manual extension retains Agent profile and Connection file selection. Claude Code and Codex configuration flows preview an additive MCP entry and preserve existing configuration. Host installation, project trust and tool approval continue to apply. MCP describes available tools; it does not itself create a chain grant.[11, 14]

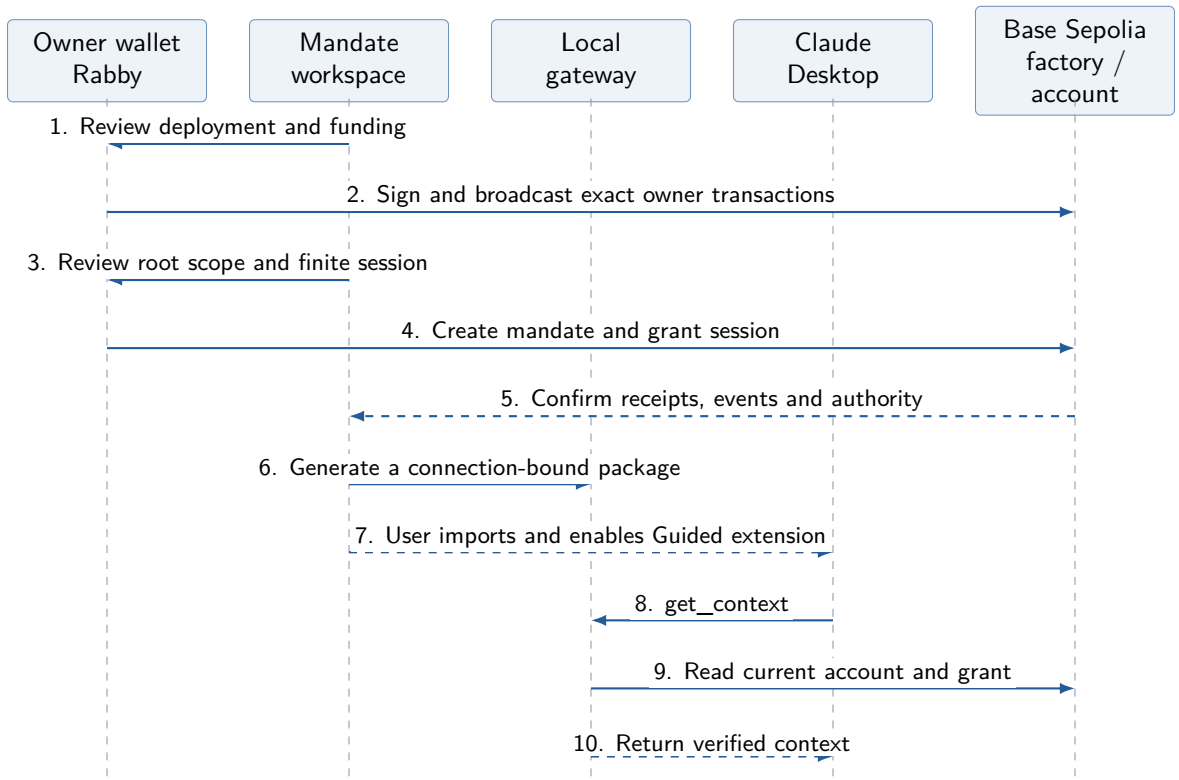


Figure 2: Account preparation and graphical agent connection. Package import is a user operation. A local MCP connectivity probe and a model invocation inside Claude are different checks. Reinstalling a connector does not reactivate a revoked grant.

2.3 Data handling and isolation

Private stores retain session/operator keys, connection configuration, payment payload signatures and raw signed transactions needed for recovery. Public exports omit those values and expose approved public-data summaries and evidence references. Mode-0600 files and restricted directories reduce accidental disclosure. They do not isolate a signer or journal from a process running as the same user or from the administrator.

The public verifier consumes an exported bundle and an independently reviewed trust configuration. The latter pins chain, account code, owner, asset, request, merchant identity and RPC assumptions. A claim inside the bundle cannot supply its own trust root. Multiple RPC observations improve cross-checking but do not constitute an independently verified consensus proof.

3 System model and security objectives

3.1 Entities, state and adversary

Fix a chain c, account W, owner O and six-decimal asset A. A root r has policy version v_r, budget B_r, maximum M_r, expiry T_r, occupancy O_r and approved merchants and steps. Session s has signer K_s, policy version, epoch e_s, budget B_s, maximum M_s, expiry T_s, occupancy O_s and a step subset. Occupancy symbols O_r, O_s are distinct from the owner identity O.

A payment p records root, policy version, session, epoch, step k, merchant, request and quote hashes, recipient, amount a_p, validity and nonce. Let C_r and C_s be registered commitments not safely released. Let $U(r, k)$ indicate a permanently used business step, and let $N(W, n)$ be the token's used-authorization state. Canonical chain transactions define transition order. A reverted operation changes no application state but may spend gas.

The hostile agent host can change prompts, call the session signer directly, forge local application state, concurrently request actions, delay or suppress submissions and delete its local evidence. It may collude with a merchant over delivery. It cannot forge the owner key or alter a correctly executing account/token outside their normal interfaces. This distinction deliberately permits financial tests that bypass the SDK. Honest SDK behavior is not an assumption of on-chain budget safety.

3.2 Assumptions

ECDSA signatures and Keccak commitments must provide their intended authenticity and binding properties. The EVM must execute transactions atomically. The approved token must enforce its authorization nonce and amount, invoke the tested ERC-1271 route at actual settlement, and not maliciously upgrade its behavior. The wallet pins the token domain, not the token proxy's future implementation. A same-domain malicious token upgrade is outside the financial guarantee.

The owner key and the owner's interpretation of the displayed administrative intent remain trusted. The selected RPCs and historical states must correspond to the claimed canonical chain. The account's immutability does not remove these external assumptions. The same-host evidence/controller implementation cannot assume its own protection from the administrator.

3.3 Objectives and non-objectives

The financial objectives are bounded root/session authorization, one business registration per step, binding of the concrete payment to approved scope, revocation at settlement, and conservative release. The accountability objective is to establish which disclosed observations correspond to that authorization and canonical payment, while distinguishing evidence integrity from independent source provenance.

Availability, timely inclusion, refunds, merchant honesty, arbitrary result quality, secrecy of public-chain amounts, global limits on unobserved model calls and completeness of an unanchored history are separate objectives. The present release does not guarantee them. A missing observation can result from a crash or outage and does not by itself prove malicious suppression.

4 Authorization objects and payment protocol

4.1 PolicySpec and the actual owner authorization path

`PolicySpec` is the specification-level view of owner intent: domain, root identifier, task hash, limits, merchant map and approved steps. In the deployed account it is represented by `Policy`, `Merchant`, `Step` and their versioned mappings. The owner signs administrative transactions, and the contract enforces `onlyOwner`. The account does not accept a generic off-chain `PolicySpec(userSig)` as an alternative administrative path. This differs from the prospective interface in the previous proposal.[7, 11]

Object	Security-relevant contents	Lifecycle constraint
Root policy	taskHash, budget, maxPerPayment, occupied, expiresAt, version, paused/revoked/existence	ID never recreated after creation; task hash fixed
Merchant	ID, payTo, optional quoteSigner	Versioned; maximum 32 per policy
Business step	ID, merchantId, approved requestHash	Versioned approval; usedStep persists across versions
Session	signer, budget/maximum/occupied, expiry, policyVersion, epoch, revoked/existence	New ID, current root subset; maximum 64 approved steps
Commitment	Full Payment plus exists and released flags, indexed by token digest	Stores authorization state, not a paid flag

Table 2: Deployed state objects, distinct from prospective research schema fields.

Creation requires a new nonzero root and task hash, valid limits, nonempty scope and unique merchant/step identifiers. Replacement raises the version, preserves occupancy and permanent used steps, and cannot lower the budget below occupancy. Pausing and resuming also raise the version, making prior sessions stale. Resuming a root therefore does not automatically restore its old grants. Root and session revocation are irreversible for their identifiers. A session must expire no later than its root, approve only current root steps, and respect both root and session amount limits.

4.2 Canonical requests, terms and deterministic nonces

The canonical request hash is

$$h_{req} = \text{keccak256}(\text{canonicalJSON}\,(\{\text{schemaVersion} : 2, \text{method}, \text{url}, \text{body}\}))\,. \tag{1}$$

The implementation recursively sorts object keys, preserves array order and uses JSON serialization for supported finite primitive values. It is a project-specific encoding, not a claim of general RFC canonicalization conformance. Amounts are bounded integer values, serialized as canonical decimal strings at JSON boundaries; floating-point procurement amounts are rejected. The URL uses the platform URL parser's normalized `href`. Credentials and fragments are rejected, GET bodies must be null, and only the supported GET/POST methods are accepted. Query parameter reordering is not automatically treated as the same approved request.

The quote hash commits to the selected canonical payment requirements. The gateway chooses only a compatible asset, network, recipient, amount and facilitator route. A changed quote is not silently substituted into an already persisted purchase. An owner-configured nonzero merchant

signer also supplies an internal `MerchantQuote` signature before registration; zero signer explicitly omits that proof rather than creating an authenticated quote.

For a root r and step k, the account and SDK derive

$$n_{r,k} = \text{keccak256}(\text{abi. encode}(c, W, r, k)) . \tag{2}$$

Session and policy version are intentionally absent from this business nonce. They remain present in the registration context. A new session or policy version therefore cannot transform a retry into a new purchase of the same step.

4.3 Typed messages and domains

The registration signature covers every field in table 3 using EIP-712 with domain name `MandateWallet`, version `2`, chain c and verifying account W.[2] The token-payment signature uses the selected token's own domain and `TransferWithAuthorization` message. Write

$$R(p) = H_{712}(\mathcal{D}_W, \text{RegisterPayment}(p)), \tag{3}$$
$$D(p) = H_{712}(\mathcal{D}_A, (W, p.to, a_p, p.validAfter, p.validBefore, n_{r,k})). \tag{4}$$

The digest $R(p)$ authorizes reservation. The digest $D(p)$ authorizes the concrete token transfer. They serve different functions and cannot substitute for each other. A merchant's optional internal signature uses the account domain and a narrower `MerchantQuote` structure binding the root/version, step, request/quote hashes, payee, value and payment expiry.

Payment fields	Solidity type	Meaning
mandateId, sessionId, stepId, merchantId	bytes32	Root, grant and business scope
policyVersion, sessionEpoch	uint64	Current authority generation
requestHash, quoteHash	bytes32	Approved request and selected terms
to	address	Approved payee
value	uint256	Positive atomic token amount
validAfter, validBefore	uint256	Token authorization bounds
nonce	bytes32	Deterministic business nonce

Table 3: Exact fields bound by the deployed RegisterPayment message.

The account accepts the tested 65-byte session ECDSA signature. It does not implement a generic multisignature or arbitrary validator-prefixed format. The token digest indexes the entire stored commitment, so the account can recover the corresponding signer and root/session context without embedding a new grant identifier inside the token signature.

4.4 Registration, signature release and settlement

A live payment requires the deployment chain, an existing active unexpired root, matching policy version, an existing unrevoked unexpired session, matching session version and epoch, and an approved session step. Registration additionally checks exact merchant/request/payee/nonce, nonzero quote hash and positive amount, unchanged token domain, unused token nonce, both per-payment limits and both remaining budgets.

The current account fixes `validAfter=0`. At registration time t, the payment expiry satisfies

$$t < p.validBefore \leq \min(t + 300, T_r, T_s). \tag{5}$$

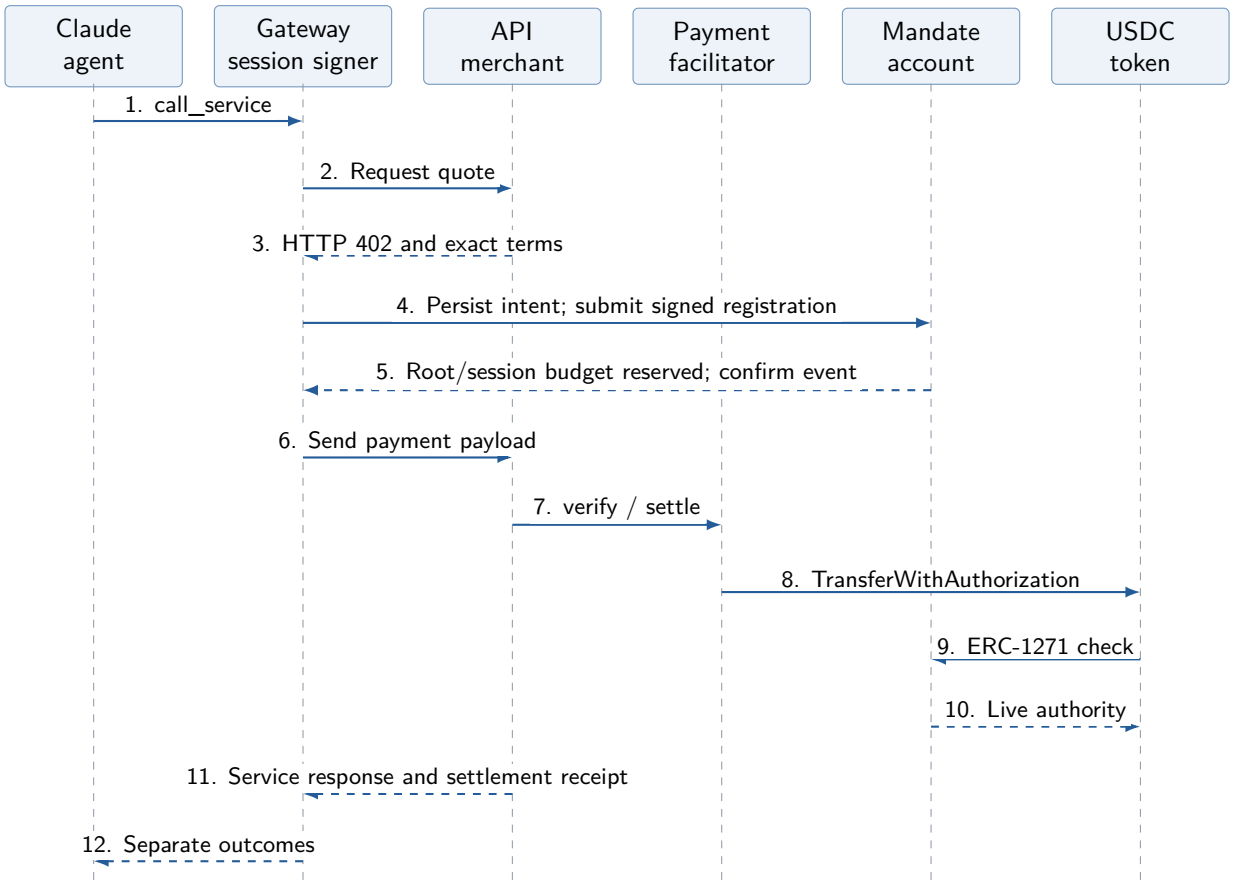


Figure 3: The deployed-account x402 purchase route. Step 6 occurs only after canonical registration. The token performs its own live wallet check at settlement. The gateway verifies the resulting token events separately from HTTP delivery and data quality.

The client may choose a shorter window for a quote. A two-hour session does not change this immutable maximum payment window. At signature validation the account requires $p.validAfter < t < p.validBefore$. Safe release instead requires $t > p.validBefore$. Equality at the deadline is not a settlement or release success condition.

Algorithm 1 Atomic registration (implementation-oriented specification)

Require: payment p, session signature σ_R, optional merchant signature σ_Q

1: Read current root, session, merchant and approved step
2: Require live version/epoch and approved request, step, recipient and asset domain
3: Require $\neg U(r, k)$, unused token nonce, positive value and valid time window
4: Require $a_p \le M_r$, $a_p \le M_s$, $a_p \le B_r - O_r$, $a_p \le B_s - O_s$
5: Verify σ_R over $R(p)$ with the approved session signer
6: If quoteSigner is nonzero, verify σ_Q over the internal merchant quote
7: Require no commitment already exists at $D(p)$
8: Atomically set $O_r \leftarrow O_r + a_p$, $O_s \leftarrow O_s + a_p$, $U(r, k) \leftarrow$ true
9: Store commitment at $D(p)$ and emit PaymentRegistered

The gateway persists the original intent and registration operation before broadcast. It checks the canonical registration receipt and corresponding commitment before generating the token authorization signature. At token settlement, `isValidSignature(D(p),signature)` returns the ERC-1271 magic value only for an existing unreleased live commitment, an unchanged token domain, an unused token nonce and the correct session signer.[4] Validation is a view operation. It cannot safely serve as the place to update shared spend counters during the token's signature callback.

The selected facilitator may preverify the payload and later submit settlement. Time, revocation or policy replacement can change between those actions. A successful precheck never locks the earlier state. The actual token call must re-evaluate the account. The route depends on the particular token implementation supporting that callback; the generic transfer-authorization specification does not establish support for every token.[5, 20]

4.5 Conservative budget accounting and safe release

For each root and session, the account maintains

$$O_r = \sum_{p \in C_r} a_p, \qquad O_s = \sum_{p \in C_s} a_p, \qquad 0 \le O_r \le B_r, \quad 0 \le O_s \le B_s. \tag{6}$$

Paid commitments remain in these sets. Pending, uncertain and canceled-or-consumed-unknown authorizations also remain occupied. Only an existing, unreleased commitment whose window has strictly expired and whose token nonce is unused can leave the sets. Release subtracts the same amount from both ledgers exactly once and preserves the used-step flag.

This accounting is intentionally conservative. The token's used-nonce bit does not identify whether an authorization paid, was canceled or has insufficient observed evidence. A view callback also cannot maintain a reliable account-side paid counter for a facilitator that calls the token directly. The client therefore proves paid value from canonical token events and keeps it separate from occupancy.

Reservation is not escrow. An owner withdrawal can reduce spendable cash while occupancy remains unchanged. Different roots can collectively authorize more than the wallet's balance, and each chain has an independent ledger. Failed inclusion or service delivery does not automatically refund budget, and gas is accounted for separately from token procurement.

4.6 HTTP request binding and compatibility boundaries

The deployed token authorization contains financial fields, not the service URL or request body. Registration links its digest to the approved request hash. The controlled merchant completes the binding by checking that its actual HTTP request and the received payment correspond to that stored commitment. An arbitrary merchant that does not perform this check cannot inherit an on-chain guarantee about the semantic purpose of an HTTP request.

x402 v2 uses the payment-required, payment-signature and payment-response headers for the relevant request/response steps.[19] The implementation checks its configured route rather than accepting every scheme advertised in a catalog. Permit2, ERC-7710, counterfactual account deployment during settlement and arbitrary third-party facilitator combinations remain unsupported here. Bazaar supplies discovery metadata and cannot add a merchant or step to owner authority.[18]

5 Durable execution and recovery

5.1 Persisted identity and concurrency control

The purchase key is the tuple (c, W, r, k). The same task step therefore resolves to the same stored order even when a process restarts or another session tries to execute it. The SQLite purchase row stores its canonical JSON and a version. A lease table records the worker and expiry. Updates verify the live lease and expected purchase version within an immediate transaction, preventing a stale worker from overwriting a later result. These local controls coordinate honest workers; they are not the mechanism enforcing the chain's root budget.

The signed-transaction table stores transaction ID, chain, sender, nonce, intent hash, raw bytes, expected hash and broadcast state. A uniqueness constraint on $(chain, sender, nonce)$ prevents a second different local intent from reusing that transaction nonce. Raw bytes are private recovery material, persisted before broadcast. The public export exposes only their non-secret metadata and transaction hashes.

The audit log retains transitions. Seen quotes bind policy version, request hash and requirements hash, so a retry does not silently accept changed terms. Private result bodies and payment payloads stay out of default public tool exports. Service and result observations remain separate from transaction journal state.

5.2 Outcome state and canonical reconciliation

A payment state can be draft, registering, reserved, pending, paid, consumed_unknown, canceled, rejected or released. Delivery independently records not_requested, returned, failed or unknown. Result inspection records unchecked, valid or invalid. These axes prevent the common error of treating an outer transaction receipt, an authorization nonce or an HTTP success as proof of the entire purchase.

For paid status the reader requires a successful canonical receipt, sufficient configured confirmations, a token AuthorizationUsed event for the exact wallet/nonce, and the matching Transfer from the wallet to the approved recipient for the exact amount. A used nonce without that evidence remains consumed_unknown. An absent receipt remains unresolved while RPC observations are uncertain. The reader checks receipt block hashes and compares configured providers; it cannot infer that a missing response means the transaction was never sent.

A transaction timeout preserves the original intent, raw operation, nonce and expected hash. Bounded rechecks and explicit read-only reconciliation precede recovery. A transport error cannot

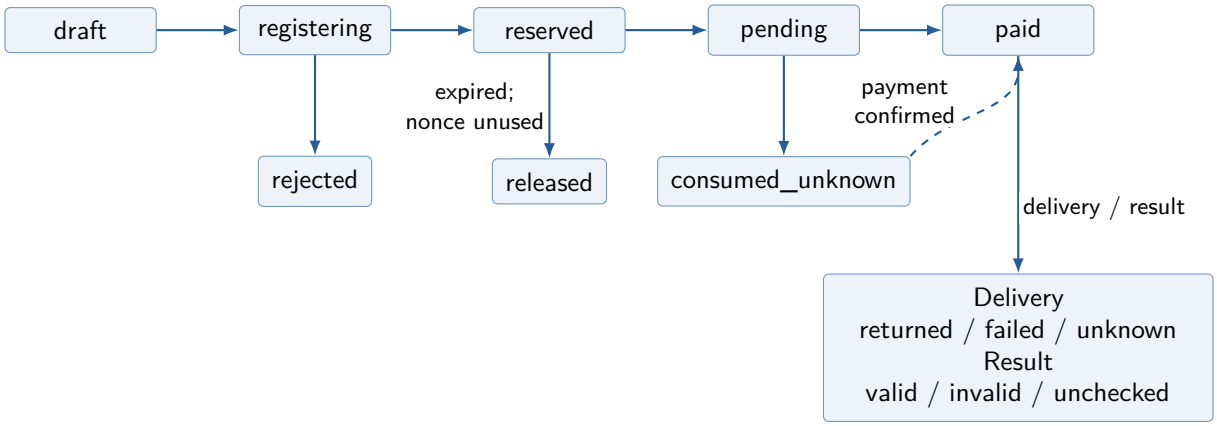


Figure 4: A projection of the durable purchase state machine. Payment, service and result are separate axes. The implementation also retains canceled token status and failed/unknown observations. A reconciliation edge is a state investigation, never permission to issue a new payment.

authorize a new business nonce, refreshed quote or replacement payment window. A reorg can remove a previously observed registration or payment. Recovery re-evaluates canonical state and, where valid, restores the same signed operation. New outcome checkpoints are ordered after the recovered canonical payment, while earlier observations remain in the journal.

5.3 Paid failure, expired commitment and administrative recovery

If the token payment is proven but service delivery fails, the client retrieves the same paid order. The controlled merchant checks its cached result against canonical settlement before returning it. An unchanged hash alone is insufficient if its receipt became noncanonical. The public demonstration exercises HTTP 500 after payment followed by HTTP 200 without a second settlement.

An unpaid payment commitment that expires is a different case. It must not be signed with a later expiry or a fresh nonce. A safe release may remove occupancy after the strict expiry/unused-nonce check, but the business step stays used. A genuinely new purchase requires a new owner-approved root or step. During recording, an earlier registered order expired before settlement; the exact transport cause could no longer be recovered from its old generic diagnostic. The owner approved a new ETH task in the same funded account. The successful task's later paid-service recovery reused its own nonce and hash. These two histories are retained separately.[8]

Owner actions also have durable identity. The application distinguishes requested, signed/broadcast, canonically applied and currently active authority. A transaction can have applied successfully yet its session later expire or be revoked. Guide progress reset, a connector reload and rotating a connection credential are not substitutes for an on-chain policy change.

6 Evidence protocol and independent verification

6.1 Data objects and stage association

A version-2 SessionAttestation is a signed checkpoint header containing chain/wallet scope, root and session, policy version and epoch, step, sequence, stage kind, previous checkpoint digest, Merkle field root, leaf count, observation time and signer. Allowed stage kinds are pre-payment,

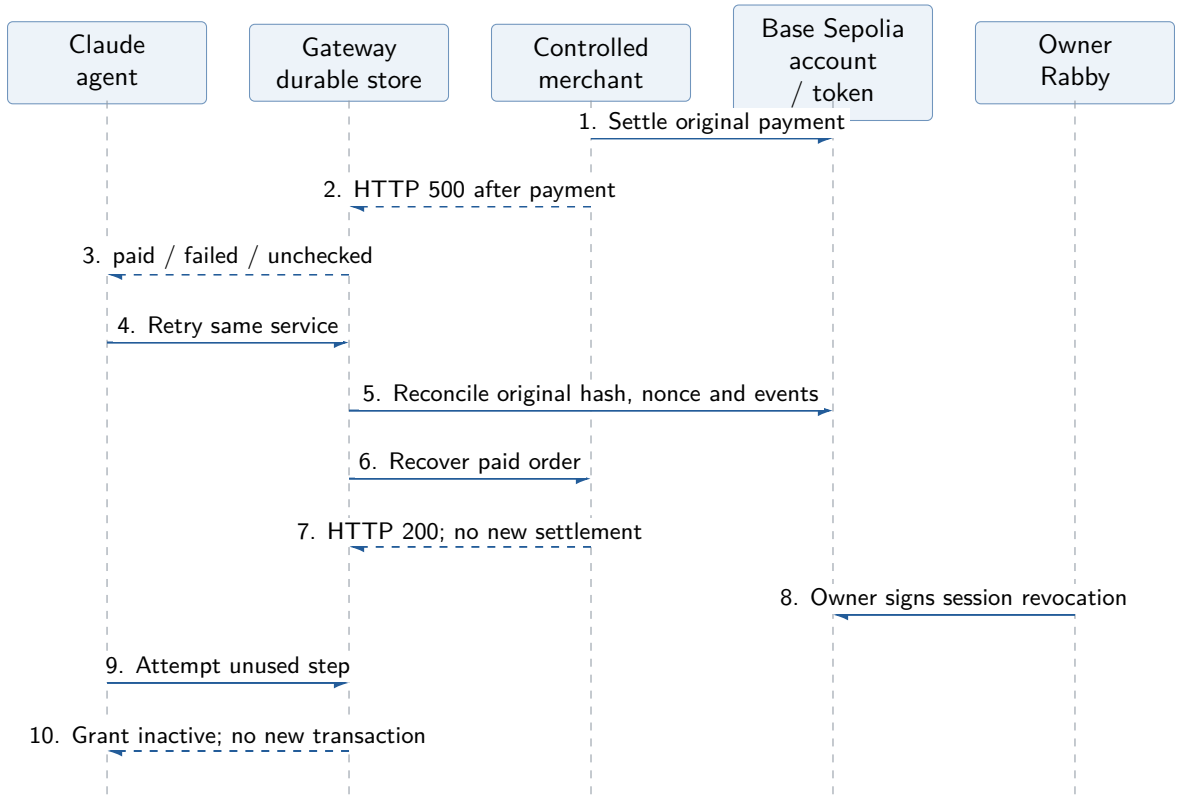


Figure 5: Observed delivery recovery and subsequent owner revocation. An expired, never-settled commitment requires a different handling path, as described in the text. The video preserves that earlier history separately.

registration, payment, service and decision. Observation time is a signed local statement, not an independently trusted clock.

The pre-payment fields bind the canonical request, terms, signed offer where present, internal quote signature, payment, token domain, registration/token digests and authority snapshot. Registration fields add the original pre-checkpoint root and transaction/block identity. Payment fields record canonical settlement identity and state. Service/decision fields record independent payment, service and result states, result digest, optionally disclosed result, merchant receipt and safe error code. The exact field inventory is in `mandate/evidence/stage-fields.ts`.[11]

ActionProof signs the shared scope, step, pre-payment checkpoint digest, $R(p)$, $D(p)$ and signer under domain `MandateActionEvidence`, version 2, chain and wallet. It links a claim about a session to a specific purchase. It is not a zero-knowledge proof, binary execution proof or on-chain verification of the complete transcript.

6.2 Canonical salted field commitments

Each checkpoint commits to a sorted set of named fields. For field index j, name n_j, value x_j and random 32-byte salt s_j, the exact construction is

$$\ell_j = \text{keccak256}(\text{abi.encode}(d_L, j, n_j, \text{keccak256}(\text{utf8}(\text{canonicalJSON}(x_j))), s_j)), \quad (7)$$

$$\text{Node}(a, b) = \text{keccak256}(\text{abi.encode}(d_N, a, b)), \quad (8)$$

$$\text{Pad}(j, m) = \text{keccak256}(\text{abi.encode}(d_P, j, m)), \quad (9)$$

where d_L, d_N, d_P are hashes of distinct versioned leaf, node and padding labels. Leaves use lexicographically sorted unique field names and consecutive indices. The tree pads to the next

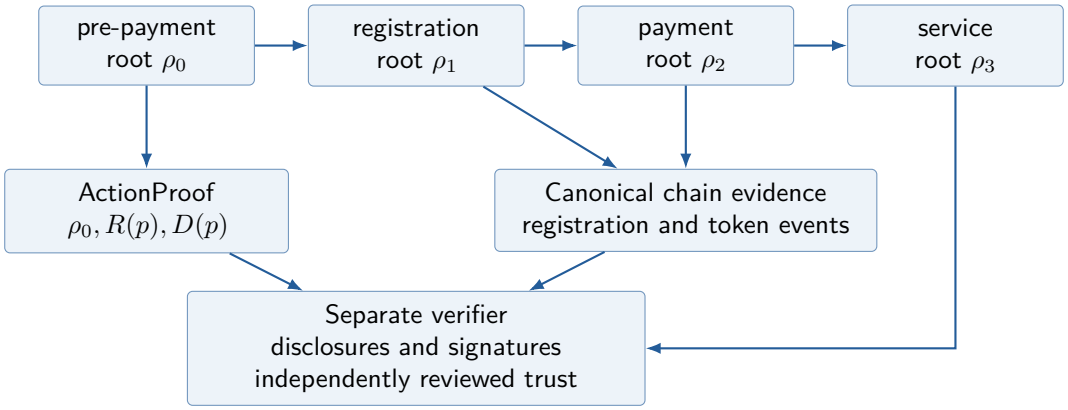


Figure 6: Evidence association. Each ρ_i signs a header containing a separate Merkle field root μ_i and the previous checkpoint digest. ActionProof links the pre-payment checkpoint to registration and token-payment digests. The account does not verify the entire off-chain checkpoint chain.

power of two with explicit padding hashes. Including order, field name, salt and separate domains avoids conflating values, structure and padding.

If m fields produce field root μ_i, checkpoint i has digest

$$\rho_i = H_{712}(\mathcal{D}_E, \text{typedHeader}(scope, k, i, kind, \rho_{i-1}, \mu_i, m, t_i, K_s)), \tag{10}$$

where $\mathcal{D}_E$ is `MandateSessionEvidence`/2 bound to chain and wallet. The first previous-root value is zero. The signature authenticates ρ_i; it is distinct from the internal Merkle root μ_i. The verifier checks consecutive sequence, unchanged scope/version/epoch, the previous-root link and nondecreasing stated observation times.

A disclosure contains name, value, salt, index, leaf count and ordered siblings. The verifier requires an index in range, the exact expected number of siblings, the expected leaf count, and no duplicate index/name. The path reconstructs μ_i, and the signed header binds that root to its context. A standard disclosure costs $O(\log m)$ sibling hashes, while the full field tree construction is $O(m)$ hash work apart from sorting.

The implementation bounds a checkpoint to 1–256 leaves, an individual encoded value to 65536 bytes, total values to 1048576 bytes, nesting depth to 32 and visited JSON nodes to 10000. A disclosed path has at most eight siblings. The checkpoint-chain input has its own maximum of 10000 entries. These limits are rejection boundaries for parsing and resource use, not security claims derived from asymptotic notation.

6.3 Verifier algorithm and evidence grades

The verifier runs separately from the agent's private runtime and requires an explicit trust file. It first checks structural limits, scope, signatures, selective disclosures and ActionProof links. It then checks pinned code/owner/asset context, exact registration transaction and event, historical root/session/merchant/step authority, and occupied budget reconstructed from root registration/release evidence. It verifies the canonical token authorization and matching transfer, then evaluates disclosed merchant claims and the configured result predicate.

Historical reads at registration prevent a later revocation from incorrectly invalidating evidence that was authorized at the time. Conversely, a historic active grant does not imply permission to pay now. Budget auditing must include root-creation evidence from an appropriate start block. The current verifier bounds its audit range to 20000 blocks; an older or larger history needs a separately designed observation/import strategy rather than a claim of unrestricted archival verification.

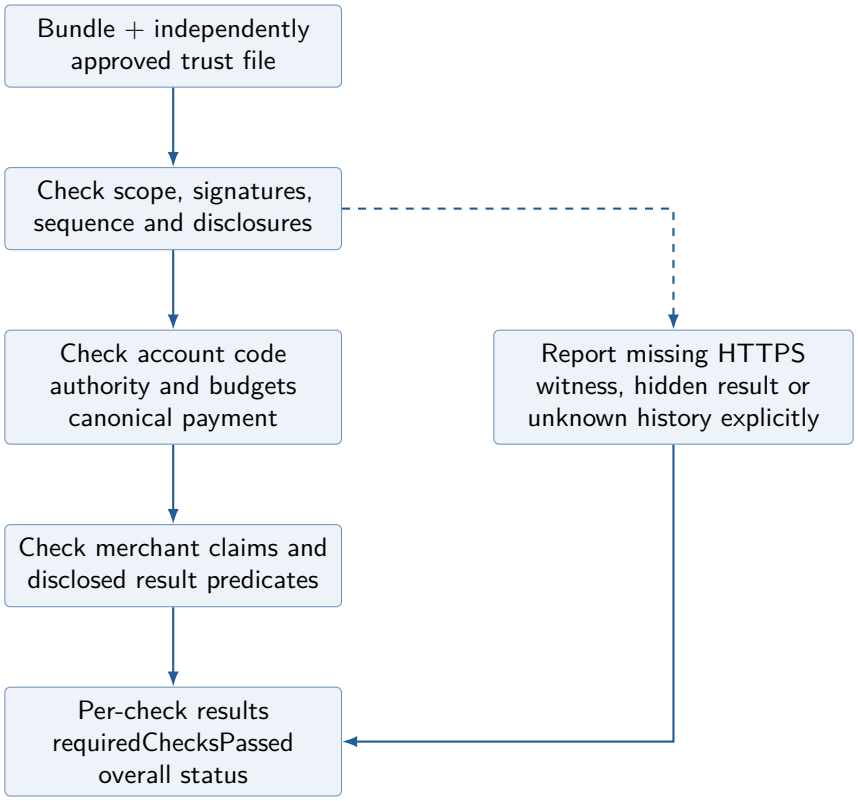


Figure 7: Independent verification flow. Required financial checks can pass while the overall report remains incomplete. A contradiction or unsupported route prevents treating the report as confirmed.

Each check reports confirmed, incomplete, contradicted or unsupported with its evidence source and exact guarantee. The required financial set comprises checkpoint integrity, ActionProof association, registration authority, budget and payment. The flag `requiredChecksPassed` also requires no contradicted/unsupported check. Overall status aggregates all checks. Therefore required checks can pass while overall remains incomplete for missing HTTPS source or unobserved history.

The merchant's standard signed offer authenticates its signed payment terms. Its standard receipt authenticates the signed resource, payer and settlement claim, with configured identity binding. The implemented receipt does not cover payment amount, request body or result body; those facts require separate evidence.[21] A locally signed checkpoint cannot independently establish that a legacy HTTPS server sent the data. The optional TLSNotary public-API experiment demonstrates a different source-evidence primitive under its own pinned notary assumptions. It is not evidence for the controlled paid merchant.[17]

6.4 Selective disclosure and privacy interpretation

Minimal export omits content while retaining the committed financial association. An auditor can verify a disclosed leaf and its context, but cannot validate a hidden request or result body. Salted commitments reduce simple direct dictionary testing when salts remain hidden; they do not provide differential privacy or a zero-knowledge compliance proof. Amount, payee and timing remain visible on chain, and low-entropy or contextual information may still be inferred.

A useful disclosure design therefore states which claim requires each field. Payment evidence needs the commitment, domain, registration and transfer context. A deterministic sample-quality claim additionally needs the result and checker definition. Independent HTTPS-origin claims

additionally need an external source witness. Whole-history claims additionally need trusted retention and observation assumptions. Reducing export bytes alone does not measure semantic privacy.

7 Security properties and host-interference analysis

7.1 Budget invariants and concurrent requests

Property 1 (Conservation and bounded authorization)**.** *Under the assumptions in section 3, successful account transitions preserve eq. (6). Token value settled through the root's commitments is bounded by its occupied budget and the owner's currently approved root budget.*

Manual inductive argument. Creation initializes occupancy to zero. Registration checks both remaining budgets and atomically increments both ledgers while inserting one commitment. Safe release removes one unreleased commitment and subtracts the identical amount once. Payment settlement, pause and revocation do not decrement occupancy. Replacement cannot set a root budget below its current occupancy. These are the only relevant transitions. Token nonce uniqueness and refusal to release consumed authorizations keep settled value within the unreleased set. Reverts preserve the prior application state. □

The result concerns this account/asset route, not all possible owner-approved roots, native gas or future malicious token behavior. Independent balance/reservation invariants exercise the implementation, but the argument has not been machine checked. The original documented property set and its counterexamples are retained separately.[10]

Property 2 (Shared-budget concurrency)**.** *With root budget 150000 and two independent 100000-unit registration requests, at most one can successfully reserve under that root before an owner-approved change or eligible release.*

The EVM orders both registrations. The second reads the occupancy resulting from the first and fails the remaining-budget condition. This holds even if both prior off-chain simulations succeeded and the requests use different transaction senders. A client-only check against the same stale balance provides no such shared-state guarantee.

7.2 Replay, version changes and signature binding

Property 3 (Business-step uniqueness)**.** *Within a root identifier's lifetime, a step registers at most once and its deterministic token nonce authorizes at most one transfer under the assumed token behavior.*

The account changes $U(r, k)$ only from false to true. Replacement, session changes and expiry release never reset it. Equation (2) also omits session/version. The token independently consumes the authorization nonce. Distinct owner-approved step IDs can still describe the same real-world service, so this property is not a semantic deduplication oracle.

The registration digest binds the entire payment context, chain and wallet. Altering amount, payee or token validity changes the token digest, which no longer indexes the original commitment. Altering root/session context invalidates registration or liveness. Scope is checked during registration, and current version/epoch must still match at settlement. A zero configured merchant signer deliberately removes merchant quote-signature authentication; it does not remove the financial constraints.

7.3 Revocation, expiry and release ordering

Property 4 (Settlement-time revocation)**.** *If revocation or replacement becomes canonical before the token's signature check for an unpaid authorization, the previous commitment cannot pass the account's live-authority predicate.*

The token's actual callback reads current root/session state. A prior facilitator precheck cannot override the new version or epoch. If payment is canonically earlier, later revocation does not reverse it. The preverify–revoke–settle experiment tests this ordering directly; the public video additionally demonstrates a fresh step rejected before broadcast after revocation.

Safe release requires strict expiry and an unused token nonce. It preserves financial conservatism at the cost of potential long-lived occupancy when cancellation or settlement evidence is ambiguous. A design that releases on HTTP failure alone would permit overspending after a payment succeeded but delivery failed. The same case motivates separate payment, service and result states.

7.4 Session evidence integrity and the missing-tail limit

Property 5 (Disclosed-field integrity under the stated signer trust)**.** *A valid disclosure and signed checkpoint bind the disclosed field, index and scope to that checkpoint, subject to the commitment and signature assumptions.*

This result authenticates a signer's statement and its linkage. A hostile session signer can still create a new internally consistent false observation. It does not establish independent source truth or a complete history.

Observation 1 (Unanchored-prefix indistinguishability)**.** *Let $H = (h_0, \ldots, h_n, h_{n+1})$ be a valid signed chain and let $H' = (h_0, \ldots, h_n)$. An auditor whose entire input is H' and who has no independent latest-head information receives identical input in a world ending at h_n and a world that suppresses h_{n+1}. Signature and previous-root checks alone cannot distinguish those worlds.*

A separately retained latest root can contradict a shortened export. It does not prove that all activity before observation or outside the retained interface was captured. The B3/B4 experiments distinguish altering disclosed data, deleting a known retained tail and deleting an unknown unanchored tail. Treating all three as the same attack would overstate the guarantee.

7.5 Reference controller, grinding and suppression

The reference controller accepts an owner-signed finite envelope binding scope, steps, a per-step query cap, start/end times, expected checkpoint kinds and deadline. Authenticated request IDs are durable and idempotent. The controller charges a new admitted query before invoking the configured query function, so a crash does not create a free query retry. It signs allow/reject/pending/completed/failed/detected statements with its own role key. A repeated request ID must match its prior digest and cannot create a second distinct query.

A cap of three rejects the fourth distinct request through the honest controller. It does not bound other model or API calls made by the same hostile host. A session-key holder can bypass the controller and still make a financially approved payment; the on-chain reservation constraints remain. A reference controller on the same machine therefore supplies a measurable interface boundary, not an enclave or malicious-administrator defense.

Missing expected checkpoints after a configured deadline produce a signed alert. They can indicate interruption, cancellation or delay, not necessarily an intentional attack. Live pre-action gating

Attack class	Current mechanism	Residual boundary
Replay	Permanent used step, deterministic nonce, original transaction recovery	Different owner-approved business IDs are distinct authority
Delay	Finite token/root/session windows and live settlement checks	Cannot force timely inclusion or delivery
Grinding	Durable query count at the reference-controller interface	Hidden calls or direct session-key use bypass the interface
Evidence suppression	Signed links and comparison with known retained roots	Unknown tail and common-host deletion remain unresolved

Table 4: Implemented controls and limits for the proposal's host-interference taxonomy.

and historical observation/import are separately labeled modes. Current financial validation does not enforce the complete controller statement or session root.

Future anti-grinding work could commit a decision before a specified future beacon round and verify the beacon signature and interpretation.[3] A public beacon alone does not prove that a host ran one model invocation or revealed every output. A future design must specify commitment timing, selective aborts, independent enforcement and liveness costs. The present experiments do not implement this path.

7.6 Research scope and remaining work

The previous proposal envisaged a more integrated proof-consuming execution pipeline. The current ActionProof is an evidence association; the account does not verify the complete SessionAttestation or ContainmentDecisionLog as a mandatory condition. The current controller uses typed statements with an envelope, request ID, kind, outcome, code, observation time, controller identity and details. This implements the decision-log role without claiming byte-for-byte conformance to every prospective proposal field. These distinctions preserve a testable connection to the research while avoiding a claim of completion for its entire agenda.

8 Evaluation design and measured results

8.1 Questions, configurations and trial controls

The evaluation asks whether the implementation preserves financial limits when clients compete or bypass safeguards, recovers uncertain service outcomes without duplicate settlement, correctly grades disclosed evidence, and exposes the cost and limits of the reference controller. The product-scoped protocol is `mandate-http-r3` with seed `0x40271015`. Local trials use chain 31337, a controlled token, merchant and facilitator, and fresh storage/chain state. Baseline order rotates by case and repetition.[9]

B1 is the project's ordinary EOA x402 reference, including honest local aggregate spend controls and durable business idempotency. B2 uses Mandate financial constraints without session-evidence association. B3 adds linked checkpoints and separate-process verification. B4 adds the live same-host reference controller.

All four HTTP groups contain 30 tasks. The 120 tasks eventually reached paid/returned/valid and produced 132 settlements with zero duplicate payments. The count includes the two-payment case. This is eventual success after explicit recovery, not first-attempt success. Each Mandate group retains three registration-signer transaction-nonce conflicts. Every group also retains the injected 500 and response-loss observations.

Cases	Fixture condition	Expected distinction
N01–N06	ETH/BTC at 1000, 10000 and 100000 atomic units	Normal exact purchase at three prices
N07	Two agents, separate ETH/BTC steps	Two payments share root authority
N08	Repeated same business step	One settlement despite duplicate calls
N09	HTTP 500 after settlement	Paid state survives failed delivery; recover original order
N10	Service processing followed by response loss	Reconcile original payment and retrieve result

Table 5: Ten normal-case definitions, each repeated three times per HTTP configuration.

8.2 Gas and latency measurement

Group ($n = 30$)	Gas/task P50 / P95	Purchase ms P50 / P95	Full-task ms P50 / P95
B1 EOA	84922 / 152732	26.81 / 45.58	28.02 / 47.46
B2 Mandate	572438 / 1110640	54.84 / 101.12	59.36 / 108.12
B3 evidence	572438 / 1110640	80.06 / 137.53	337.25 / 625.65
B4 controller	572438 / 1110664	125.49 / 229.98	416.55 / 775.76

Table 6: Recorded local HTTP results. Mixed P95 includes multi-payment tasks. Source: the frozen arithmetic/data audit.

For sorted measurements $x_{(1)}, \ldots, x_{(n)}$, the audit uses nearest-rank $q_p = x_{(\lceil pn \rceil)}$. Per-case samples have $n = 3$, so no stable population tail-latency inference follows. Full-task time includes post-checks and, where applicable, full export and a separate verifier process. Setup and secondary probes are excluded. Nested phase timings are not additive. These local times are not Base Sepolia latency, throughput or a production SLA.

The reservation transaction increases cost relative to the EOA reference. B2–B4 have the same account execution path and median gas in this experiment because evidence/controller additions run off chain. Small per-trial differences reflect calldata byte costs. Their larger full-task latency measures additional record and verification work. The result is a costed safety/evidence tradeoff, not a gas-saving claim.

B3 and B4 each produced 33 payment evidence bundles. Full/minimal byte medians were 22574/20962 for B3 and 29489/27877 for B4. Minimal disclosure retained required financial checks while leaving hidden result checks incomplete. The byte decrease measures serialization disclosure, not anonymity or a universal privacy score.

8.3 Explicit invalid inputs and stateful attacks

The invalid-call generator constructs 20 strata of 500 distinct calldata samples. Categories cover zero/excessive value, wrong recipient/request/merchant/step/nonce/quote, forbidden validAfter, expired/overlong windows, stale version/epoch, missing root/session, wrong signer/domain, session-subset violations and exhausted root budget. The generator uses a fixed xorshift32 stream plus domain-labeled Keccak values, preserves generation order, and checks the raw typed encoding against the contract digest rather than relying only on SDK input validation.

The audit verifies distinct calldata, an explicit invalidity predicate, expected error selector and exact source/seed replay for all 10000 samples. All rejected, and six legal control calls accepted. These are `eth_call` observations, not 10000 mined attacks or random draws from every possible adversary strategy. Repeating the same seed is reproducibility, not a second independent sample.

Eight core/boundary stateful cases cover shared-budget competition, session subsets, same-step replay, preverify/revoke ordering, version replacement, uncertain occupancy and direct/bypass boundaries. Evidence tampering, retained-tail omission, unknown-tail omission and the controller cap have separate observations. Protocol categories and executed core cases are counted separately rather than labeling the same financial scenario three times as independent coverage.

8.4 Clean reproduction and review scope

The current product regression covers the Mandate account, procurement prototype, SDK, services, evidence verifier, Web, operations, onboarding and desktop integration. The 23 account cases, 21 prototype cases and 148 TypeScript tests passed, along with build/typecheck and the local paid-failure/recovery/revocation demo. Two account fuzz suites ran 10001 cases each, and the account reservation invariant performed 8192 calls without a handler revert. The source reproduction record identifies the exact committed snapshot and independent checkout used for verification.[12]

The reproduction used declared host toolchains and public caches. It did not copy personal runtime, keys or databases. An initial sandbox npm-resolution failure is retained separately from the successful same-lockfile network-enabled retry. Later document refinements are separately hashed and do not retroactively change the recorded video source.

The automated regression and source-verification results apply to the tested snapshot. They do not constitute a zero-alert security certification. External human audit and theorem-prover verification remain incomplete.

9 Public Base Sepolia execution case

9.1 Roles, account and scope

The native run used chain 84532 with account `0xa794DAa86ff10920B75a4de3b0E752586e8703D2`. Its owner was `0xaA987fC286434D205669814Ba46b1d8df824fB36`, signing through Rabby. The asset was native Base Sepolia test USDC at `0x036CbD53842c5426634e7929541eC2318f3dCF7e`. The session signer was `0x818D3bd3ddf805FA9916AB50813d0EB67e40862f`, and the payee was `0x5B608Ea8219185dc5C971D2924913bF1e60fCEc1`. The owner key was not supplied to Claude or the production gateway.[8]

The root was `0x65a5386f35ae1dc848584bd51d1a8211110aa02b8c5264238aa4d4aa7d5a8c89`. The two-hour session was `0x0263e35063f897a6639f0272e8d6b8a3ff3d19c772f5d8b04baf05ae7c4ebd47`. These are historical identifiers, now subject to expiry and confirmed revocation. They are not reusable live access credentials.

9.2 Transactions and separate outcomes

The account received 150000 atomic USDC units, independently of native gas funding to the execution role. The successful task registered and paid 10000 atomic units once. Payment nonce `0xa65dbb50c51b68982cb7c19d440efad2bf574303856b0bb48a949f82f7cedddb` and settlement hash remained unchanged during paid-service recovery. Final observed wallet balance was 140000 units. The settlement and revocation rows below are the principal explorer evidence; all eight rows have exported receipts and logs.

Table 7: Eight key public transactions. Hashes are clickable Base Sepolia explorer links.

Purpose	Block	Transaction hash
account	48316879	0x2ba48a2f3197c6dfac2b3c5579b0e3e01350a2ba1290d019d044a685ed21cb26
fund gas	48318701	0x6ddf321b74a123e0ac7ee5007042c80dd667ac25a993f790622c258a91aebe2e
fund usdc	48318779	0x3bb4b38d34ddea128a261e8c5da4ca28dfd2369f8671f2b4a718ed8c25ff61e2
task	48324013	0x8336dc072584e2c29201de51e6d7ee1d44ba5c0ddf475188abcee7f0973b7505
session	48324110	0x6311ac19a4b212aa656d2e8b74a631f5c99fc0a109d3bfdf273c93ab142a1180
registration	48324434	0xdad95b5ada48d537a8fcd117be8ebab442d2032b4c1ac18810d5dd9d77d8b1e1
settlement	48324438	0xc23d943b69c94a5a8468ab32ee9b454ab6b57f7681ba4b93f16e4d58ae840c5d
revocation	48324929	0x5ba3feb1febde1fc2637c524ac031aa17e0a9f583bf39fc843bb865162d69a6e

The service outcome first recorded paid/failed/unchecked with HTTP 500, then paid/returned/-valid with HTTP 200. The valid result means the disclosed fixed sample matched the explicit deterministic checker. It does not establish an accurate market observation. After owner-signed session revocation, Claude attempted the unused BTC sample and received `AGENT_GRANT_INACTIVE`. The step remained unregistered, no service was requested and no new transaction or USDC transfer occurred.

The public evidence retains an earlier expired, unsettled ETH order and an explicitly authorized new task in the same wallet. The wallet was not redeployed and no second top-up was hidden as recovery. An updated connector version 0.2.1 and its installed bundle fingerprints are also recorded. The raw run came from a working tree with changes; its file and bundle hashes, rather than Git HEAD alone, identify its source.

9.3 Independent evidence status and finality interpretation

The standalone verifier report confirms checkpoint integrity, request disclosure, ActionProof, registration authority, budget, canonical token payment, merchant offer/receipt and fixed-result checks. It has `requiredChecksPassed=true` and `overall=incomplete`. Independent HTTPS source evidence and complete history are missing. The workspace button lacked a trusted verification profile during recording. The successful report came from the standalone verifier with the explicit trust file, not from that button.

Two RPC providers reported all eight transactions below their common finalized height of 48325041 at the recorded observation. The exported receipts preserve block hashes and fee components. Provider agreement is a trust-based observation, not a locally verified consensus or rollup proof. The code and evidence are available for read-only review without a signer or new payment.

The picture edit contains actual UI captures with cuts, crops, scaling and held views. It is not an uninterrupted or synchronized multi-feed recording. The current picture duration is 290 seconds, with user narration pending in the material preparation snapshot. The merchant and facilitator

remain controlled fixtures. No arbitrary Bazaar, mainnet, signed-installer or independent audit acceptance follows from this run.

10 Release boundaries and next implementation work

The current product provides contract-level financial scope, a graphical owner workspace, source-mode onboarding, MCP integrations, recoverable service procurement and independently graded evidence. The public native-agent case and the controlled adversarial experiments support specific claims about this implementation. They do not justify treating the previous research proposal's entire intended protocol as deployed.

The next engineering priorities are signed installation and updates, persistent local practice recovery, remaining operating-system/wallet usability paths, stronger isolation of session signing and retention, and broader tested merchant/facilitator compatibility. A genuinely independent paid-merchant HTTPS witness and externally managed latest-head retention require additional deployment and trust design. Beacon-bound decision commitments, zero-knowledge execution evidence and ERC-8004 integration require separate specifications and experiments.

The task checklist remains authoritative: the original T0–T11 set retains 97/105 completed subitems, T12 retains 7/8, and G3 remains open in this snapshot. The new document does not change runtime code, authorize mainnet spending or constitute an official submission. Academic programme/year information on the title page is the submitted registration information, not a newly inferred biography.

A Contract and MCP interface reference

A.1 Account methods and observable errors

Method	Responsibility and checks
`createMandate`	Owner-only creation of a new root, task, limits, merchants and approved steps.
`replacePolicy`	Owner-only new version with occupancy/used-step preservation.
`setPaused`	Owner-only pause/resume with version increment and stale old grants.
`revokeMandate`	Owner-only irreversible root revocation.
`grantSession`	Owner-only new session ID, signer and current policy subset.
`revokeSession`	Owner-only revocation and epoch increment.
`registerPayment`	Session/quote verification, atomic double reservation and permanent step use.
`isValidSignature`	View-only validation of an existing registered digest under current authority.
`releaseExpired`	Permissionless safe release after strict expiry with an unused token nonce.
`ownerWithdraw`	Owner-only token withdrawal with SafeERC20 and reentrancy guard.
`getPolicy/getSession/getStep /getMerchant/getCommitment`	Read policy, grant, versioned scope and committed payment.

Method	Responsibility and checks
`paymentNonce/paymentDigest/registrationDigest/quoteDigest`	Shared digest/nonce encoding for clients and review.

Observable account errors include Unauthorized, InvalidPolicy, InvalidSession, InvalidPayment, InvalidSignature, BudgetExceeded, StepAlreadyUsed, UnsafeRelease and UnsupportedAsset. They identify the failed account predicate, not an automatically inferred malicious actor. ERC-1271 returns its specified valid/invalid bytes4 values for its normal signature path.

A.2 MCP tools and effect boundaries

Tool	Inputs and intended effect
`mandate_get_context`	No arguments; read bound identity, chain, root/session and budgets.
`mandate_request_authorization`	taskDescription, budgetAtomic, maxPerPaymentAtomic and finite ttlSeconds; prepare an unsigned owner request.
`mandate_search_services`	query and bounded limit; discover untrusted metadata and approved configured services.
`mandate_quote_service`	serviceId; obtain current terms and scope checks without payment.
`mandate_call_service`	serviceId; execute or reconcile the original approved purchase. This can spend test assets.
`mandate_get_task`	No arguments; inspect independent payment/delivery/result states.
`mandate_get_receipt`	serviceId; export a redacted receipt/evidence reference.
`mandate_request_revoke`	No arguments; prepare an unsigned owner session-revocation request.

Tool schemas are strict. A service ID refers to configured scope; it is not an arbitrary URL or arbitrary-sign operation. Requesting authorization or revocation does not sign, broadcast or activate it. MCP annotations are host-facing hints rather than financial enforcement. The contract and owner transaction boundary remain authoritative.

B Reproduction and deployment reference

The companion reproduction guide supplies detailed commands and acceptance interpretation. The minimal local sequence uses Node 24.x (at least 24.2), pnpm 11.25.0, Python 3.12+ and Foundry 1.5.1. The contracts pin Solidity 0.8.26, optimizer 200, viaIR and Cancun. A separate synchronized execution/consensus node is unnecessary for this reproduction.

```
python3 scripts/bootstrap-sdk.py
CI=true pnpm install --frozen-lockfile
pnpm sdk:build
pnpm sdk:demo artifacts/reproduction/demo
```

```
pnpm setup:build
pnpm setup:start
```

The demo uses local Anvil and mock USDC. The setup process prints a loopback URL, normally port 18743. Use an independent profile and never copy somebody else's bound package or private runtime. Public deployments use separately funded roles, a checked asset domain and explicit owner intents. Existing historical manifests and signing journals must not be overwritten or reused with another identity.

Read-only verification uses the public bundle and separately reviewed trust file:

```
node --import tsx mandate/evidence/cli.ts \
  --bundle competition/video/claude-desktop-base-v1/execution-evidence/purchase-bundle.json \
  --trust competition/video/claude-desktop-base-v1/execution-evidence/verifier-trust.json \
  --output /tmp/mandate-public-verification.json
```

Missing historical RPC data leaves a check incomplete. It is not a reason to create a new payment. Base Sepolia native test USDC and Ethereum Sepolia controlled-token experiments retain separate manifests. Base mainnet inspection is read-only, and Permit2/ERC-7710/third-party facilitator acceptance does not follow from protocol-level compatibility claims.

References

[1] Vivek Acharya. Secure Autonomous Agent Payments: Verifying Authenticity and Intent in a Trustless Environment. arXiv:2511.15712, 2025. URL https://arxiv.org/abs/2511.15712.

[2] Remco Bloemen, Leonid Logvinov, and Jacob Evans. EIP-712: Typed structured data hashing and signing. https://eips.ethereum.org/EIPS/eip-712, 2017. Accessed 20 October 2026.

[3] drand contributors. drand distributed randomness beacon documentation. https://docs.drand.love/, 2026. Research extension only. Accessed 20 October 2026.

[4] Francisco Giordano, Matt Condon, Philippe Castonguay, Amir Bandeali, Jorge Izquierdo, and Bertrand Masius. ERC-1271: Standard Signature Validation Method for Contracts. https://eips.ethereum.org/EIPS/eip-1271, 2018. Accessed 20 October 2026.

[5] Peter Jihoon Kim, Kevin Britz, and David Knott. ERC-3009: Transfer With Authorization. https://eips.ethereum.org/EIPS/eip-3009, 2020. Accessed 20 October 2026.

[6] Zibin Lin, Shengli Zhang, Guofu Liao, Dacheng Tao, and Taotao Wang. Binding Agent ID: Unleashing the Power of AI Agents with accountability and credibility. arXiv:2512.17538, 2025. URL https://arxiv.org/abs/2512.17538.

[7] Mengzhong Ma. Agent-Authenticated Execution: Verifiable Delegation and Host-Interference Mitigation for the Agentic Economy. Ethereum Foundation PhD Fellowship programme proposal, March 2026. Internal prior-work reference; prospective research objectives, not release evidence, 2026.

[8] Mandate Labs. Native Claude Desktop, Rabby and Base Sepolia execution evidence. https://github.com/MandateLabs666/mandate-wallet/tree/b44f8aaa8a8f57c03e982beef4625dd31b1d2ffd/competition/video/claude-desktop-base-v1/execution-evidence, 2026. Recorded 19 October 2026; public evidence, trust configuration and source fingerprints.

[9] Mandate Labs. M7 benchmark protocol, raw-data audit and acceptance. https://github.com/MandateLabs666/mandate-wallet/blob/b44f8aaa8a8f57c03e982beef4625dd31b1d2ffd/pla

nning/M7_BENCHMARK_ACCEPTANCE.zh-CN.md, 2026. Protocol mandate-http-r3; 120 HTTP tasks.

[10] Mandate Labs. M7 formal properties, counterexamples and guarantee boundaries. https://github.com/MandateLabs666/mandate-wallet/blob/b44f8aaa8a8f57c03e982beef4625dd31b1d2ffd/planning/M7_FORMAL_BOUNDARIES.zh-CN.md, 2026. Manual arguments; not a machine-checked proof. Historical task-status statements retain their original date.

[11] Mandate Labs. Mandate Wallet: tested source snapshot. https://github.com/MandateLabs666/mandate-wallet/tree/b44f8aaa8a8f57c03e982beef4625dd31b1d2ffd, 2026. Product-scoped source and exact test identity in the reproduction record. Accessed 20 October 2026.

[12] Mandate Labs. Clean-source reproduction and publication records. https://github.com/MandateLabs666/mandate-wallet/tree/codex/submission-2026-10-20/artifacts/competition/source-delivery/mandate-only-20261020, 2026. Product-scoped build, regression and independent reproduction records.

[13] MetaMask. Smart Accounts Kit documentation. https://docs.metamask.io/smart-accounts-kit/, 2026. Accessed 20 October 2026.

[14] Model Context Protocol contributors. Model Context Protocol Specification. https://modelcontextprotocol.io/specification/2025-11-25, 2025. Protocol reference dated 2025-11-25; repository pins SDK 1.27.1. Accessed 20 October 2026.

[15] Marco De Rossi, Davide Crapis, Jordan Ellis, and Erik Reppel. ERC-8004: Trustless Agents. https://eips.ethereum.org/EIPS/eip-8004, 2025. Identity-layer context, not implemented authorization in this release. Accessed 20 October 2026.

[16] Safe. AI agents powered by Safe Smart Accounts. https://docs.safe.global/home/ai-overview, 2026. Accessed 20 October 2026.

[17] TLSNotary contributors. TLSNotary documentation: Introduction. https://tlsnotary.org/docs/intro/, 2026. Accessed 20 October 2026.

[18] x402 contributors. Bazaar: Discovery Layer. https://docs.x402.org/extensions/bazaar, 2026. Accessed 20 October 2026.

[19] x402 contributors. HTTP 402 payment protocol documentation. https://docs.x402.org/core-concepts/http-402, 2026. Version 2 wire integration in the pinned implementation. Accessed 20 October 2026.

[20] x402 contributors. Wallet Compatibility. https://docs.x402.org/advanced-concepts/wallet-compatibility, 2026. Accessed 20 October 2026; actual token and facilitator behavior must be tested.

[21] x402 contributors. Signed Offers and Receipts. https://docs.x402.org/extensions/offer-receipt, 2026. Accessed 20 October 2026.